

Fine-Tuning Techniques for Transformer Models

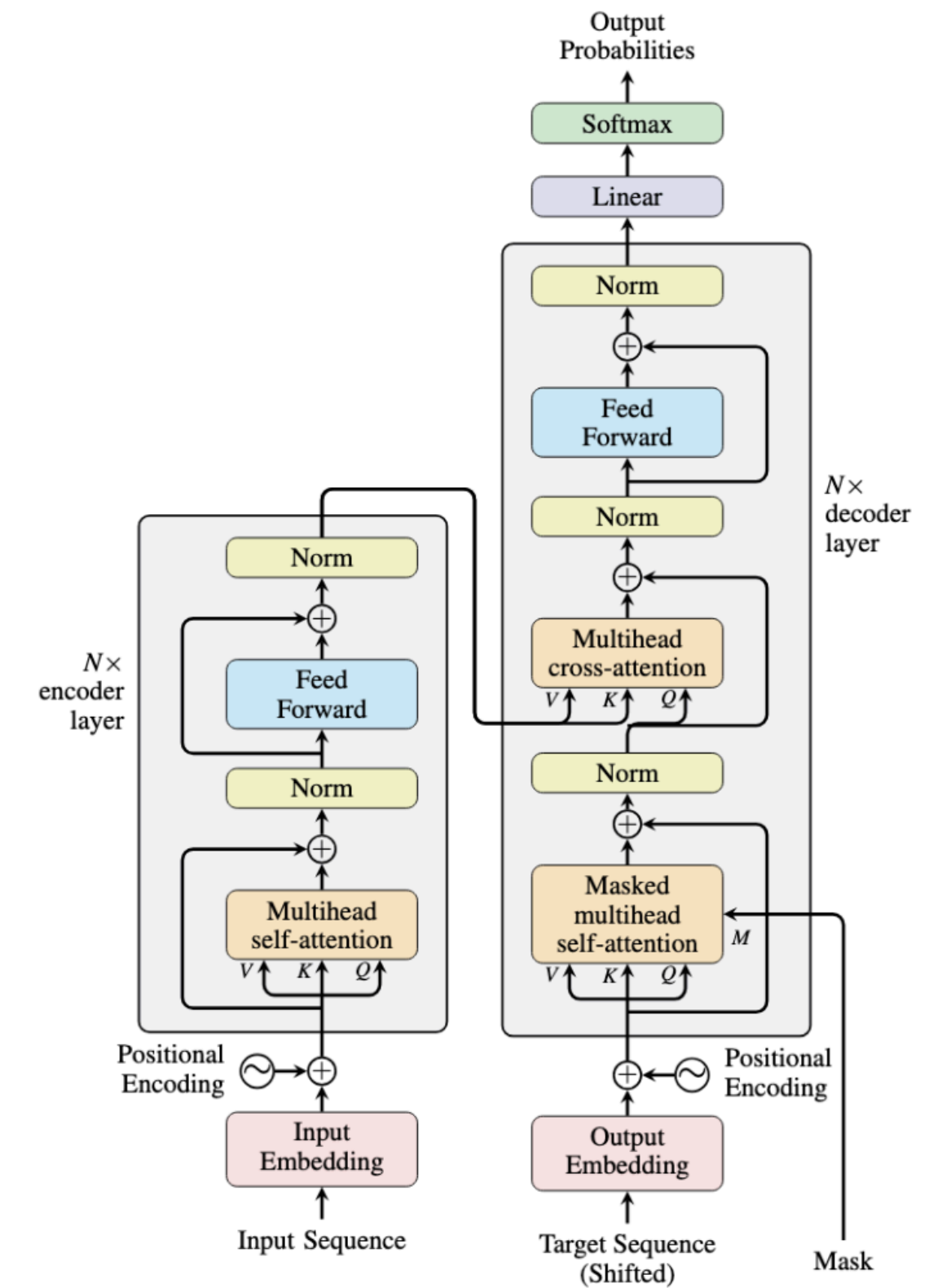
Anna Vettoruzzo

Overview

- Transformer model (recap)
- Pre-training vs. post-training
- Supervised fine-tuning
- Parameter-efficient fine-tuning
- LoRA: Low Rank Adaptation
- Advanced insights into LoRA

Transformer model (recap)

- Positional encoding
- Multiple blocks with:
 - Self-attention layer to learn interactions
 - Normalization layer to normalize the input
 - Residual connections to preserve gradients
 - Feed-forward layer to learn mappings
- Encoder-Decoder architecture

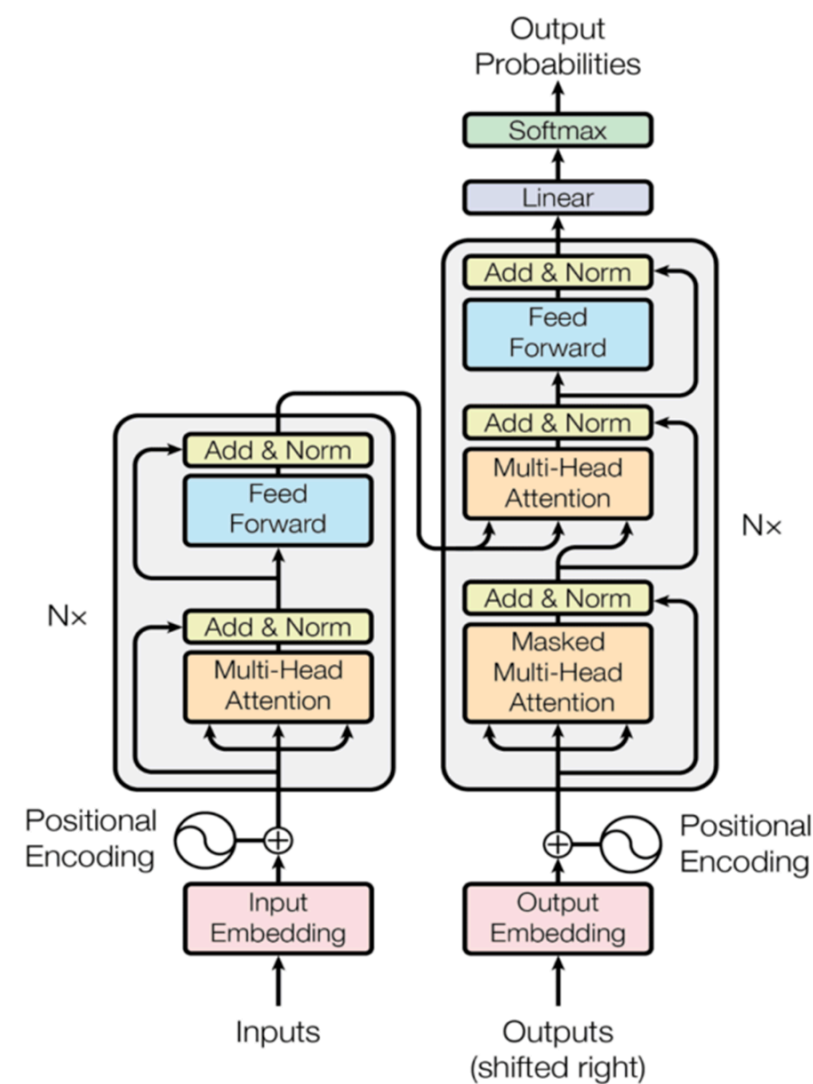


Source: <https://machinelearningmastery.com/encoders-and-decoders-in-transformer-models/>

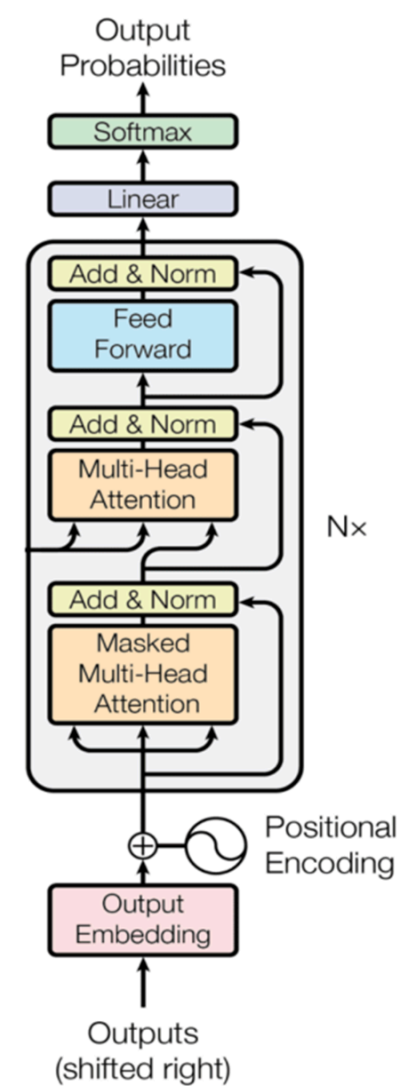
But ... there are different variations of this architecture

- Encoder-decoder models (e.g., T5) are ideal for seq-to-seq tasks like translation.
- Decoder-only models (e.g., GPT) generate text (next-token prediction). They rely on masked (causal) self-attention where a token at position i only attends to tokens at position $i-1$.
- Encoder-only models (e.g., BERT) extract sentence embeddings. To do so they use bi-directional self-attention (i.e., the representation of a token at position i is informed by tokens at position $i-1$ and $i+1$).

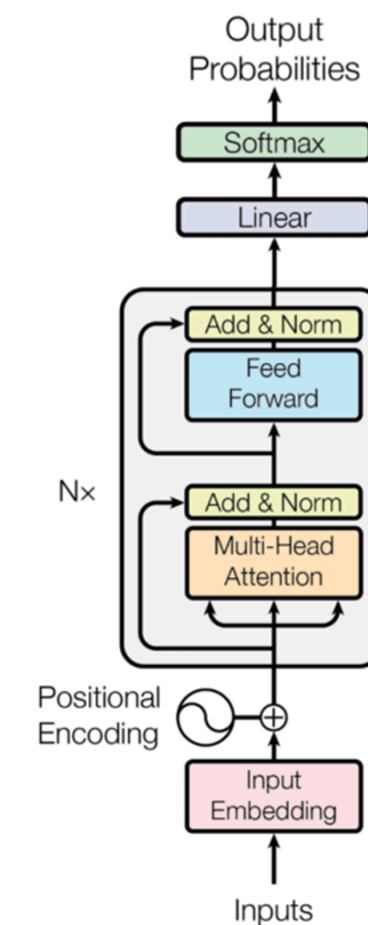
Transformer (encoder-decoder)



Decoder-only (GPT*)



Encoder-only (BERT*)

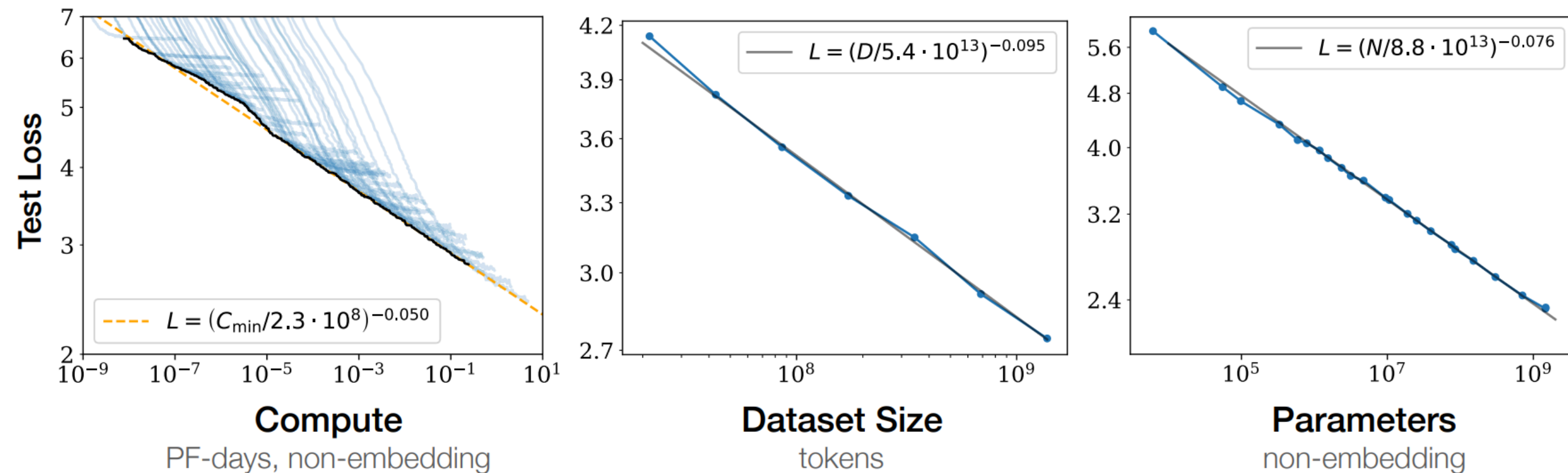


* The architecture might vary a bit.

In this lecture, we will focus on a decoder-only architecture!

Large Language Models (LLMs)

Large decoder-only models with hundreds of billions of parameters. The larger the model, the better the performance.



Source: [Scaling Laws for Neural Language Models \(Kaplan et al.\)](#)

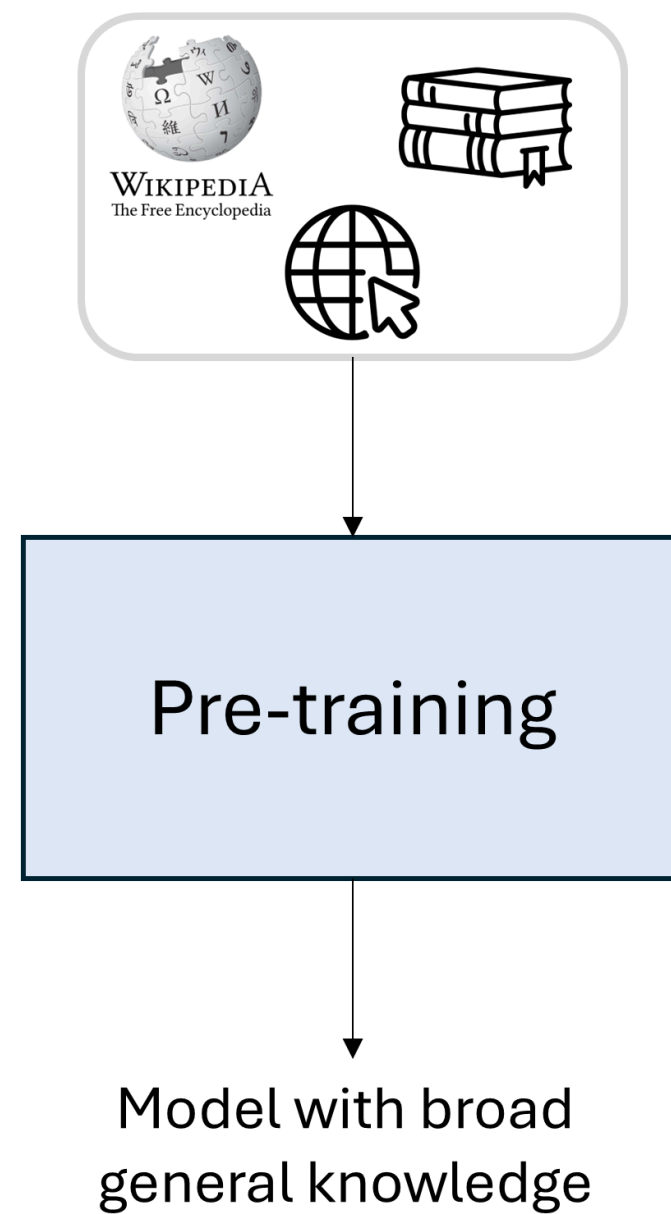
Increase the model size → Increase in the number of GPUs/time needed for training → Increase in cost. 💰

Training a model from scratch for each task is not feasible. The solution is to divide the training pipeline into two different phases: **pre-training** and **post-training**.

Pre-training

Goal: Learn language & world knowledge.

- **Input:** Massive raw datasets.
- **Outcome:** A *base model*.
- **Limit:** Only predicts the next token; doesn't "follow instructions."

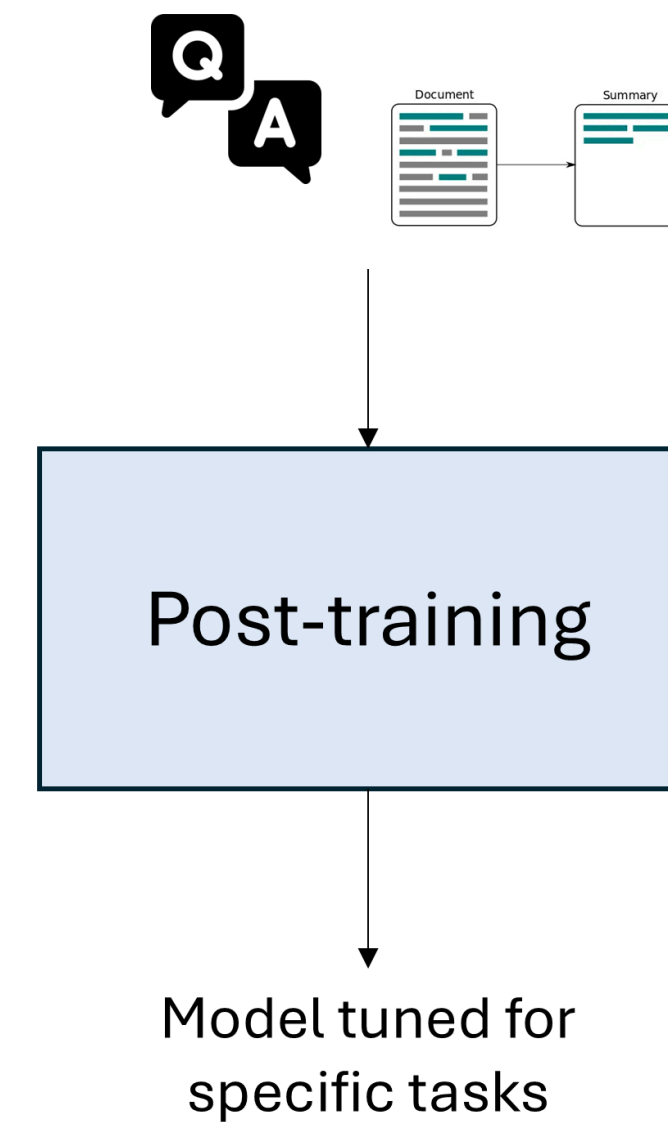


Post-training

Goal: Refining a pre-trained model to behave in a specific, useful, and safe way.

Methods:

- **Supervised fine-tuning (SFT):** Adapts model to specific tasks or domains. Outcome: an instruction-tuned model.
- **Preference tuning:** Ensures responses are in the correct format, helpful & harmless. Methods: RLHF, DPO.



So, you can see that there is a **paradigm shift** from tradition ML and LLM training:

- **Traditional ML.** We train a model for each task, or we apply transfer learning by reusing the trained model to some capacity.
- **LLM training.** The idea is to train the model to understand general knowledge, then tune it for an end task.

An example with a simple prompt (remember that the generation is randomic!)

Prompt: What is the capital of the Netherlands?

Llama-3.2-1B (base model):

What is the time zone in the Netherlands? What is the population of the Netherlands? The distance between Amsterdam and The Hague is 27 miles. What is the time difference between Amsterdam and The Hague? What is the currency in the Netherlands?

Llama-3.2-1B-Instruct (instruction-tuned model):

Amsterdam.

The Netherlands is a country located in the Northwestern Europe. It is known for its rich history, beautiful canals, and vibrant culture. The capital of the Netherlands is Amsterdam.

Amsterdam is a major city and a popular tourist destination, famous for its canals, historic architecture and vibrant nightlife. It is also home to many museums, art galleries, and cultural events. The city has a population of about 850,000 people and is a major financial and trade hub.

Some of the most popular attractions in Amsterdam include:

- The Rijksmuseum, which houses a vast collection of Dutch art and history
- The Anne Frank House, a museum dedicated to the Jewish girl who wrote

Authenticate and load the model

```
# Authentication to Hugging Face
hf_token = ... # TODO: add your HF token here
login(hf_token)

# Model configuration
# Feel free to try with a different model
base_model_name = "meta-llama/Llama-3.2-1B"

# Load models and tokenizers
base_tokenizer = AutoTokenizer.from_pretrained(base_model_name, use_fast=True)
base_model = AutoModelForCausalLM.from_pretrained(
    base_model_name,
    device_map="auto",
    dtype=torch.float16
)
```

Model generation

```
# Simple prompt
simple_prompt = "What is the capital of the Netherlands?"
# Tokenize prompt using base tokenizer
inputs = base_tokenizer(simple_prompt, return_tensors="pt").to(device)
print(f"\nPrompt = {simple_prompt}\n")

# Generate output
with torch.no_grad():
    outputs_base = base_model.generate(**inputs, max_new_tokens=150)
    print("Base model:\n", base_tokenizer.decode(outputs_base[0], skip_special_tokens=True))
```

Generation with an instruction-tuned model

```
# Better to run the code on Colab where a GPU is available for free
# Model configuration
# Feel free to try with a different model
inst_model_name = "meta-llama/Llama-3.2-1B-Instruct"

# Load model and tokenizer
inst_tokenizer = AutoTokenizer.from_pretrained(inst_model_name, use_fast=True)
inst_model = AutoModelForCausalLM.from_pretrained(
    inst_model_name,
    device_map="auto",
    dtype=torch.float16
)

# Simple prompt
inputs = inst_tokenizer(simple_prompt, return_tensors="pt").to(device)

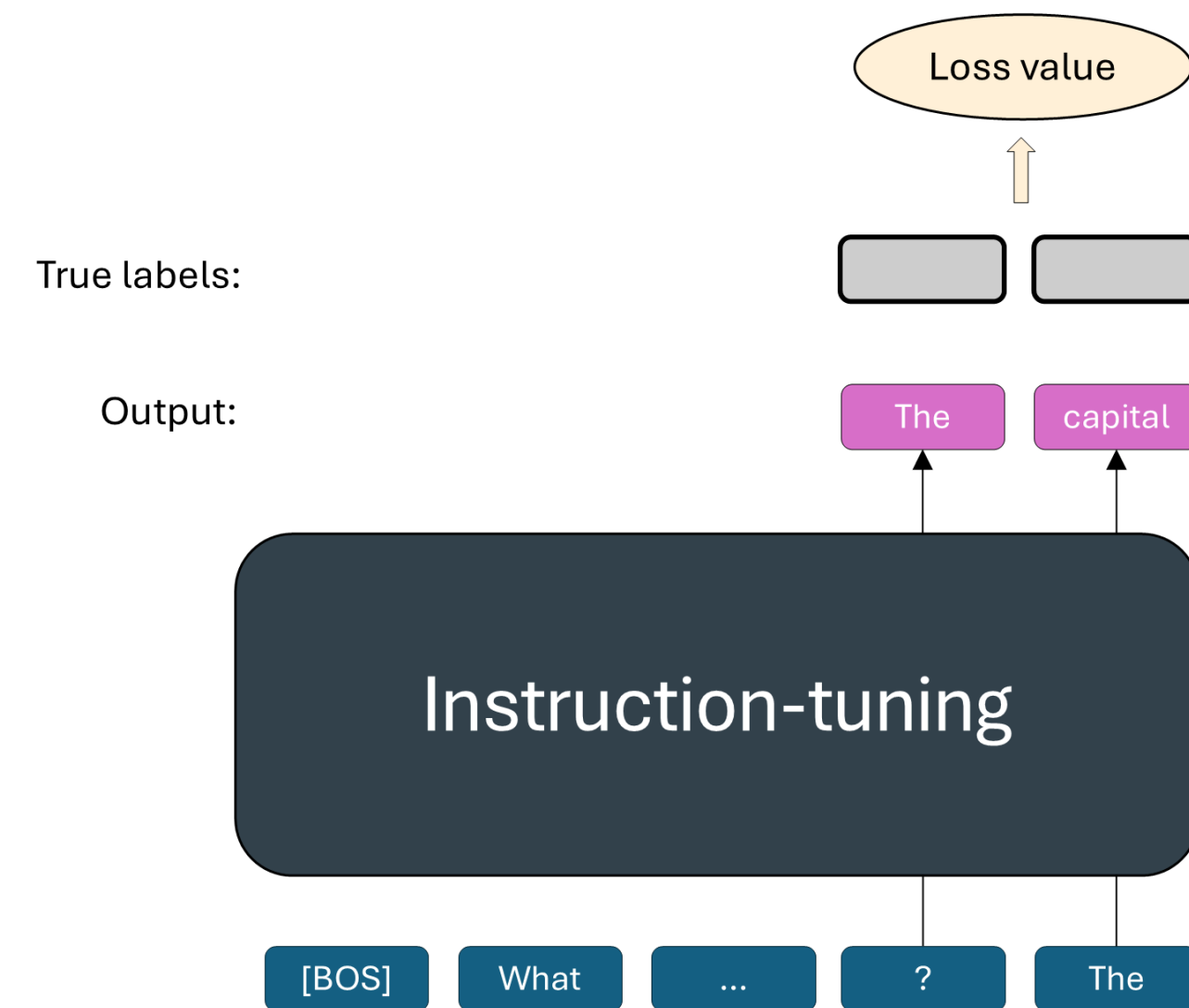
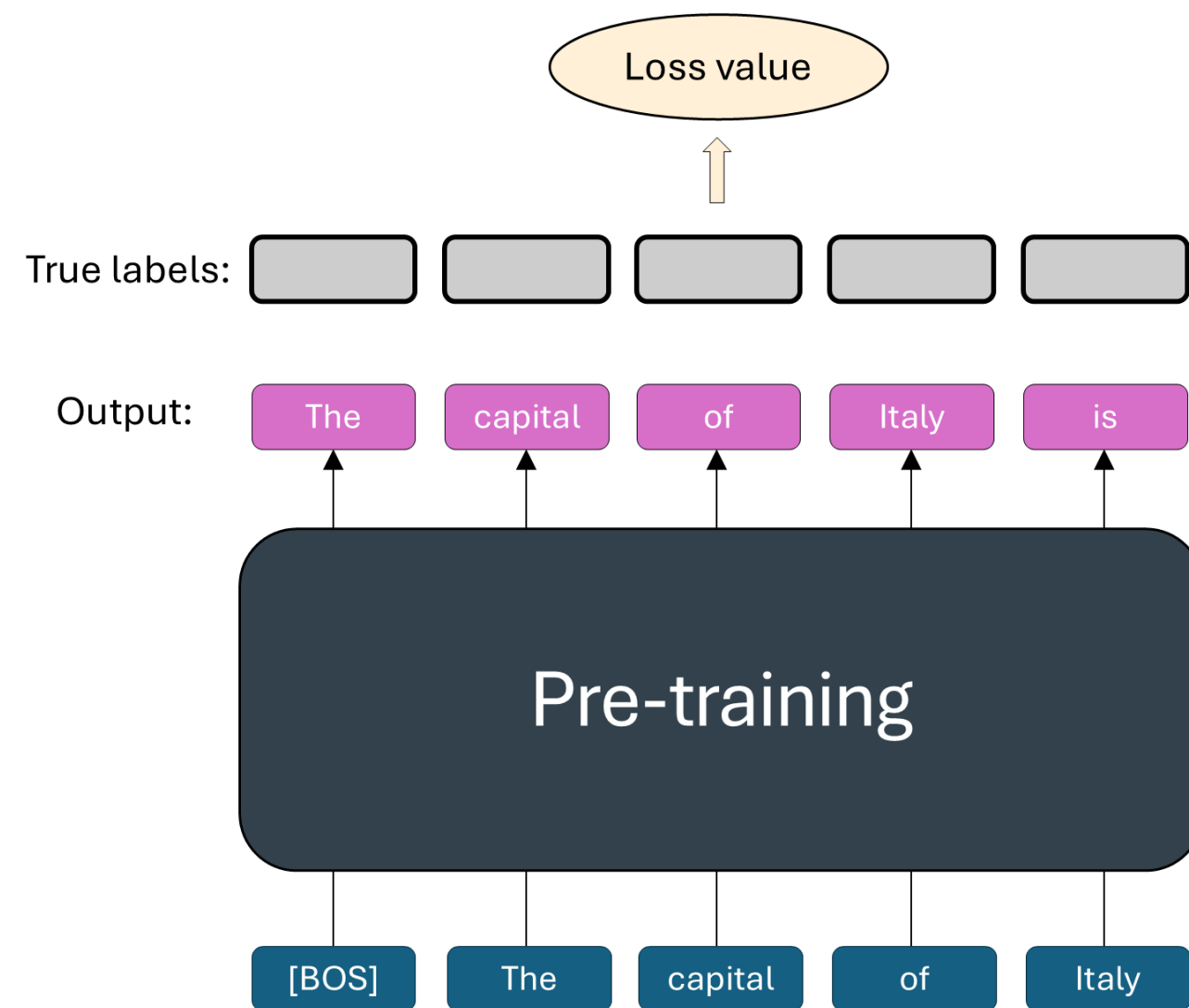
# Generate output
with torch.no_grad():
    outputs_it = inst_model.generate(**inputs, max_new_tokens=150)
    print("\nInstruction-tuned model:\n", inst_tokenizer.decode(outputs_it[0], skip_special_tokens=True))
```

Supervised Fine-Tuning (SFT)

SFT, or **instruction-tuning**, adapts pre-trained models using labeled task-specific input/output pairs to improve performance and instruction-following capabilities.

Strategy:

- Collect pairs of input/outputs with desired behavior.
- The objective function is the same as the pre-trained task (next-token prediction) but the model is given the whole input and it tries to predict the output. **The loss calculation is only on the output prediction.**
- The standard SFT approach is also called Full Fine-Tuning (FullFT) where **all model weights are updated** during the process.



Data for instruction-tuning

- High quality data
- Can be both human written or synthetic data
- It usually consists of millions or few billions of tokens vs. trillion of tokens for pre-training

Summarization Instruction: Summarize the following article in one sentence: {Article text}. Response: The article discusses the impact of AI on healthcare.	Question Answering Instruction: Why do people like comedy movies? Response: Comedy movies offer stress relief, social bonding, and an escape from reality...
Classification Instruction: Identify which animal species is alive or extinct: Palaeophis, Giant Tortoise. Response: Palaeophis is extinct, Giant Tortoise is alive.	Creative writing Instruction: Write a short story about a person who discovers a hidden room. Response: Emma traced the floral wallpaper, her fingers catching on a faint seam...

Challenges of FullFT

- Very high quality data needed as it directly determines the model's capability for the task
- Risk of overfitting on small datasets
- Need to store full model checkpoints for each fine-tuned model
- Risk of catastrophic forgetting of the pre-trained knowledge
- Computationally expensive

PEFT

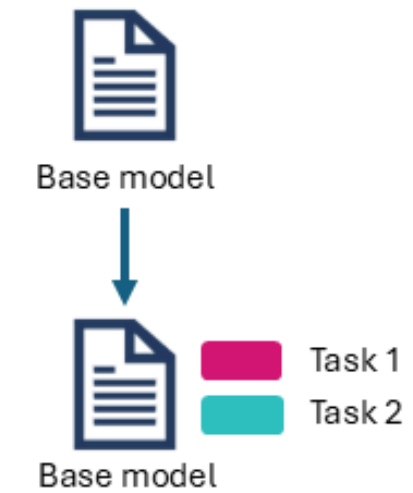
- FullFT involves updating every single weight of the model.
- Parameter Efficient Fine-Tuning (PEFT) updates only a small set of parameters, while the others remain frozen.

Full Fine-Tuning (FullFT)



- ✓ High flexibility
- ✗ Computationally intensive
- ✗ Large memory required
- ✗ Low adaptability to diverse tasks
- ✗ Longer training time

Parameter Efficient Fine-Tuning (PEFT)



- ✓ Low forgetting
- ✓ Less compute/memory intensive
- ✓ High adaptability to various tasks
- ✗ Might require choosing different hyperparameters per task

Example

Fine-tuning a 7B parameter model using FullFT can require up to 50GB of GPU VRAM and only 20GB using PEFT techniques (*actual memory usage depends on batch size, sequence length, and specific model architecture*).

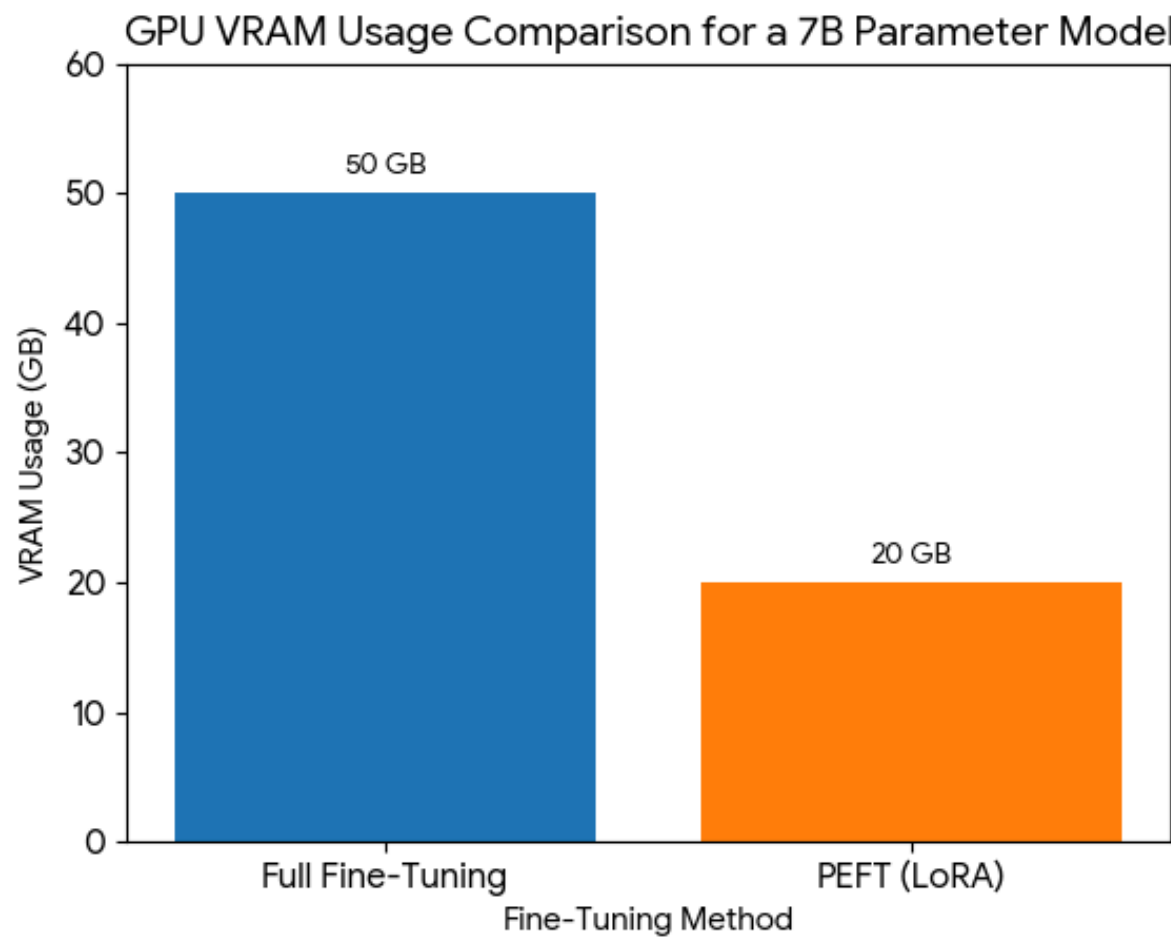
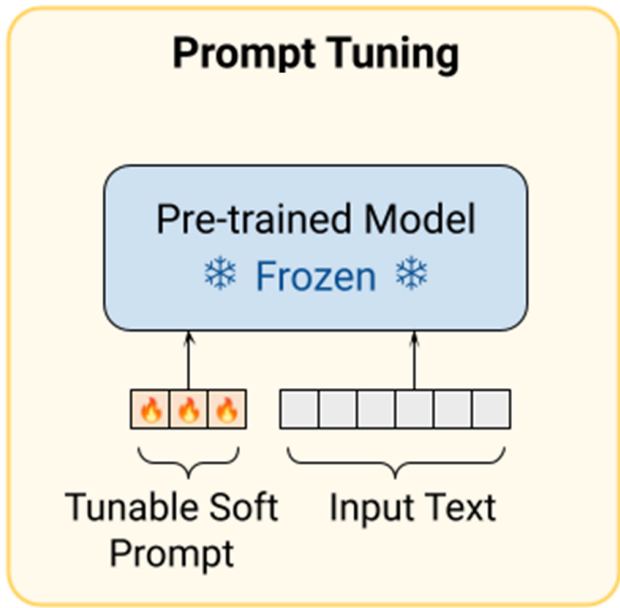


Image generated with Nano Banana.

PEFT methods

Prompt tuning

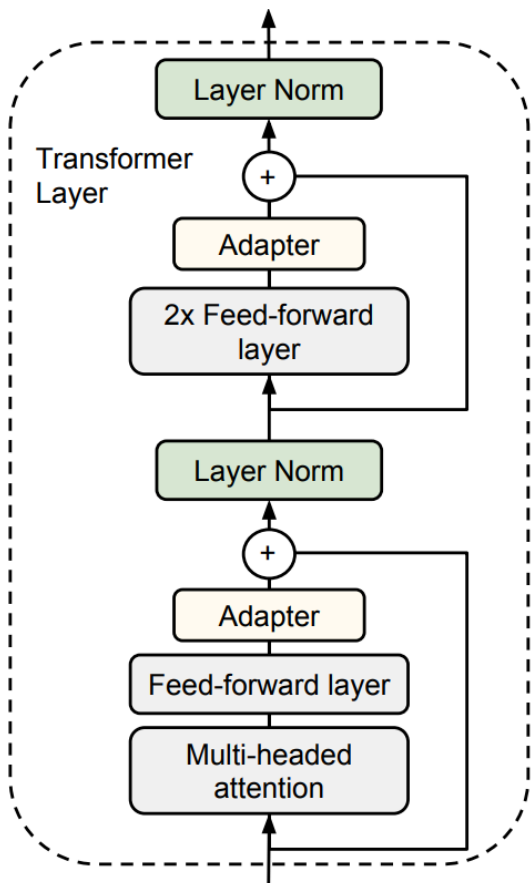
Learning "soft prompts" through backpropagation to condition frozen language models to perform specific downstream tasks.



Source: [The Power of Scale for Parameter-Efficient Prompt Tuning](#) (Lester et al.)

Adapter layers

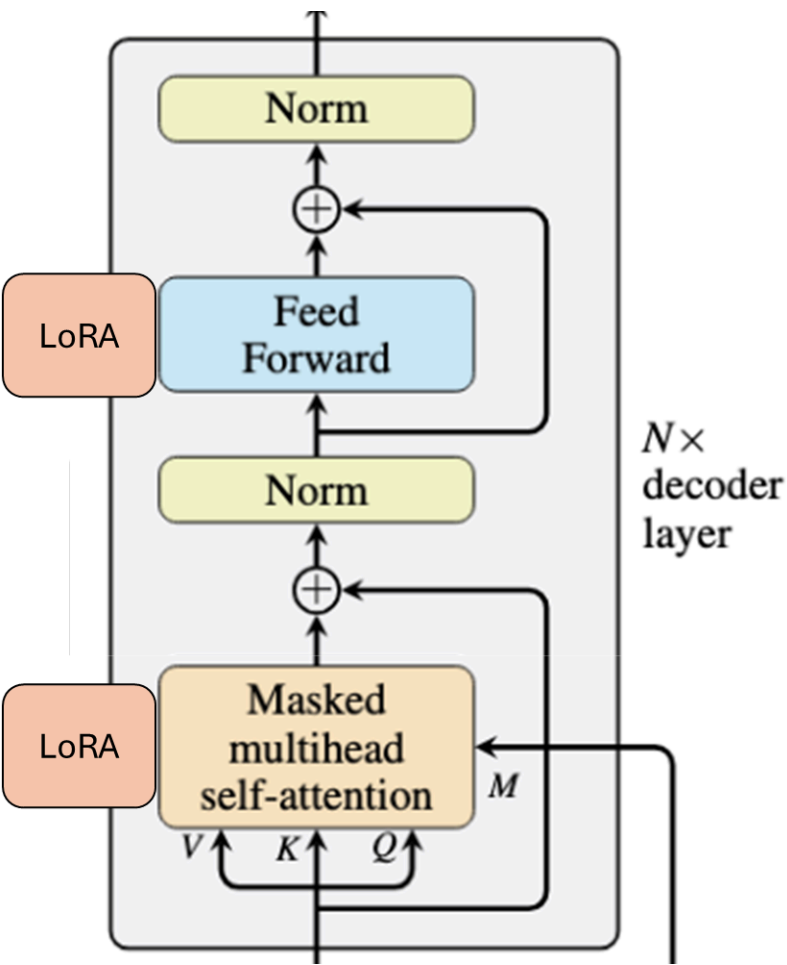
Adapters are new per-task modules added between layers of a pre-trained network.



Source: [Parameter-Efficient Transfer Learning for NLP](#) (Houlsby et al.)

LoRA

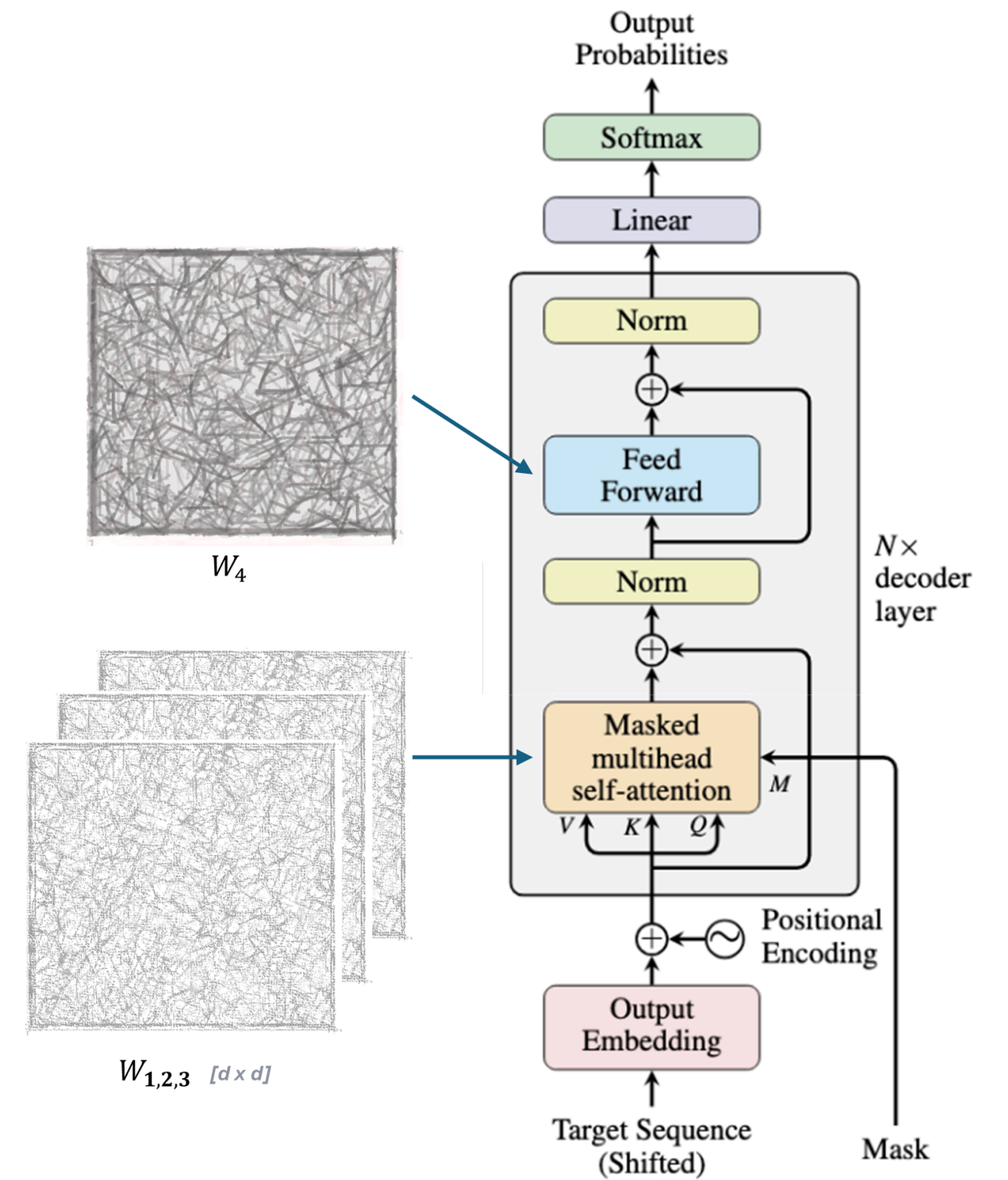
Approximate the weight updates with small tunable matrices.



Source: [LoRA: Low-Rank Adaptation of Large Language Models](#) (Hu et al.)

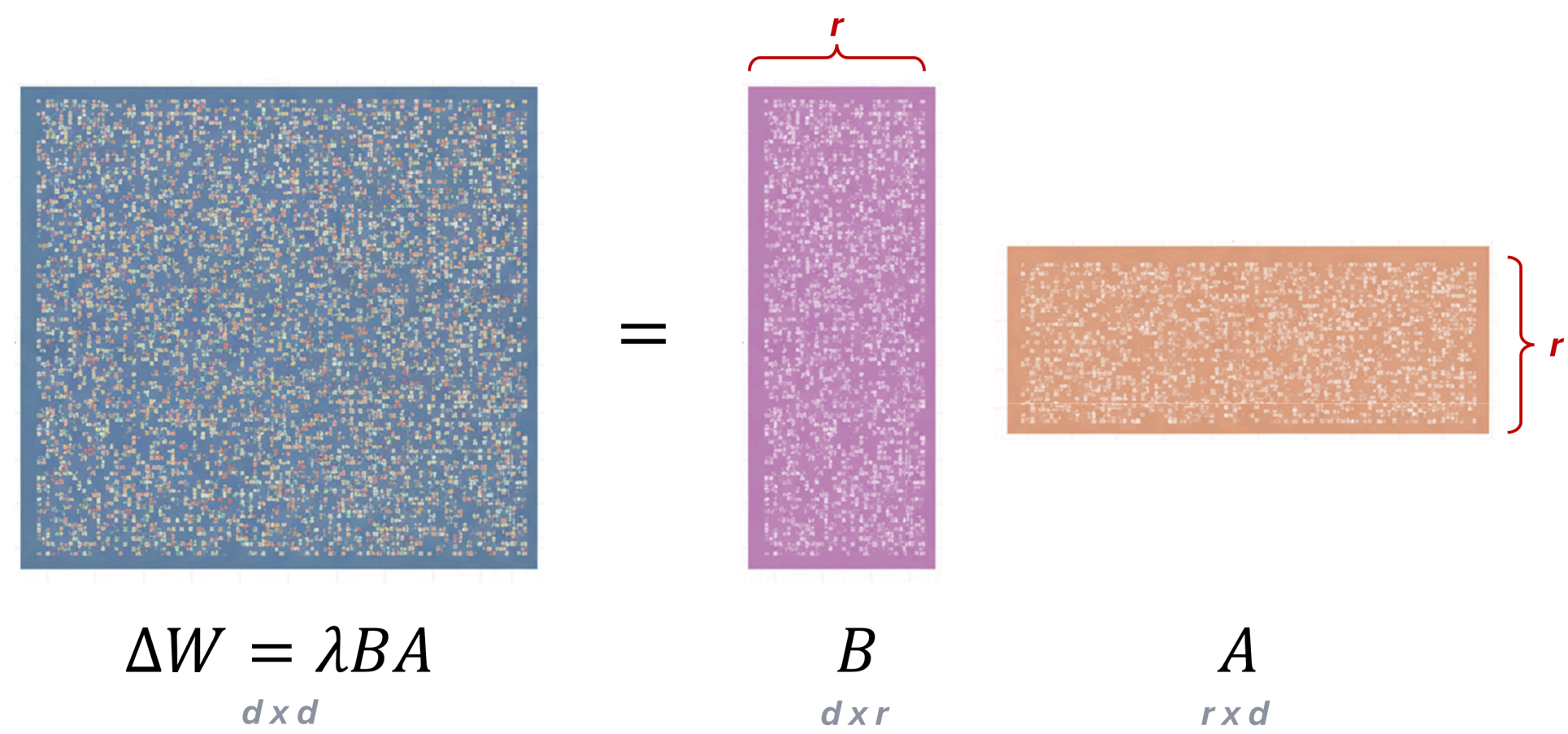
Low Rank Adaptation (LoRA)

The original weight matrix W_i of an LLM is very large.
For example, in the case of GPT-3 175B, W has a size of 12288×12288 .



Instead of learning the update for each weight $\Delta W = W_{i+1} - W_i$ during backpropagation, we can approximate it using a technique called **low-rank approximation**:

$$\Delta W = \lambda B A, \text{ where } \lambda \text{ is a scaling factor.}$$



Therefore, the fine-tuned weight matrix can be expressed as $W_{i+1} = W_i + \lambda B_i A_i$.

You might wonder how small the A and B matrices are, and this depends on the **rank r** which will see in a few slides.

Note: A and B are usually initialized as $A \sim \mathcal{N}(0, 1)$ and $B = 0$.

Gradient descent with LoRA

Mini-batch Stochastic Gradient Descent (recap)

1. Draw a batch of *batch_size* training data $\mathbf{X}$ and $\mathbf{y}$
2. *Forward pass* : pass $\mathbf{X}$ through the network to yield predictions $\hat{\mathbf{y}}$
3. Compute the loss $\mathcal{L}$ (mismatch between $\hat{\mathbf{y}}$ and $\mathbf{y}$)
4. *Backward pass* : Compute the gradient of the loss with regard to **every weight**
 - *Backpropagate* the gradients through all the layers
5. Update W : $W_{(i+1)} = W_{(i)} - \eta * \frac{\partial L(x, W_{(i)})}{\partial W}$

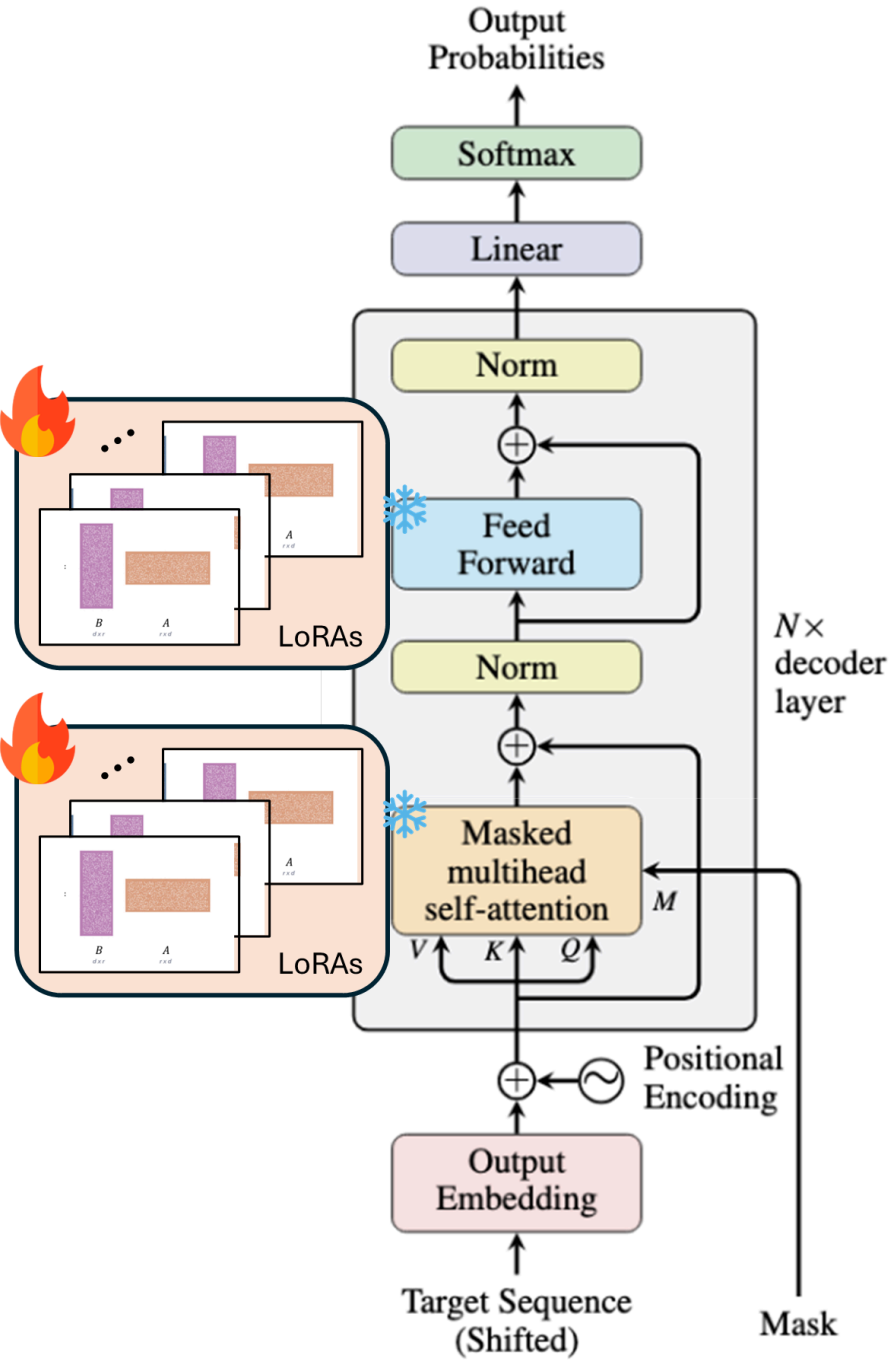
Repeat until n passes (epochs) are made through the entire training set.

With LoRA

With LoRA we need to compute the gradient of the loss with regard to **LoRAs weights**.

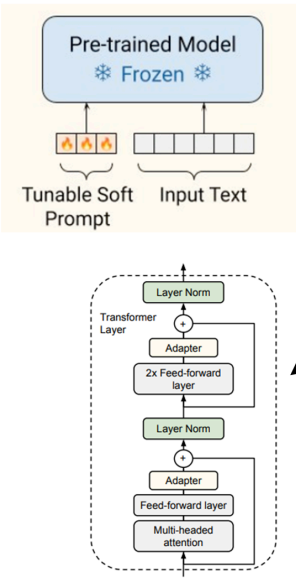
$$A_{(i+1)} = A_{(i)} - \eta \frac{\partial L(x, W_{(i)})}{\partial A}$$
$$B_{(i+1)} = B_{(i)} - \eta \frac{\partial L(x, W_{(i)})}{\partial B}$$

And the fine-tuned weight matrix can be expressed as W with $W_{(i+1)} = W_{(i)} + \lambda * B_i A_i$.



Benefits of LoRA

- Reduction in memory and storage
- Reduction of VRAM consumption
- Easy to switch between tasks, by just swapping LoRA matrices
- 25% faster training compared to FullFT
- No catastrophic forgetting, since the pre-trained weights remain frozen during adaptation



Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

Source: [LoRA: Low-Rank Adaptation of Large Language Models \(Hu et al.\)](#)

Exercise 1

How much memory does this save?

Assume an LLM has

- W with 10,000 rows and 10,000 columns $\rightarrow$ total of 100,000,000 parameters.

If we choose A and B with $r=8$,

- A has 10,000 rows and 8 columns, B has 8 rows and 10,000 columns $\rightarrow$ total $10,000 \times 8 + 8 \times 10,000 = 160,000$

625× less than 100,000,000!!

Exercise 2

How to calculate the number of trainable parameters in LoRA?

$$|\theta| = 2 \cdot L_{LoRA} \cdot d_{model} \cdot r,$$

where L_{LoRA} is the number of weight matrices we apply LoRA to.

Let's take a look at how to use LoRA in Python code.

Dataset loading

```
# Better to run the code in Colab where a GPU is available for free

# Load the dataset
dataset = "fka/awesome-chatgpt-prompts"
data = load_dataset(dataset)
data = data.map(lambda samples: tokenizer(samples["prompt"]), batched=True)
train_sample = data["train"].select(range(50))
train_sample = train_sample.remove_columns('act')
display(train_sample)
```

```
Dataset({
  features: ['prompt', 'for_devs', 'type', 'contributor', 'input_ids', 'attention_mask'],
  num_rows: 50
})
```

Model loading and LoRA configuration

```
# Load the model and the tokenizer
model_name = "bigscience/bloomz-560m" # feel free to try with other models. You can find the list of supported models here:
https://huggingface.co/docs/peft/main/en/index#supported-methods
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

# Import Lora fine-tuning configuration
lora_config = LoraConfig(
    r=4, # rank controls the number of trainable parameters
    lora_alpha=4, # a scaling factor that adjusts the magnitude of the weight matrix (lambda=a/r). Usually set equal to r
    target_modules=["query_key_value"], # the name of the modules to apply LoRA to. The name depends on the model used
    lora_dropout=0.05, # to avoid overfitting
    bias="lora_only", # this specifies if the bias parameter should be trained
    task_type="CAUSAL_LM"
)

peft_model = get_peft_model(model, lora_config)
print(peft_model.print_trainable_parameters())
```

Train LoRA

```
# Train the model
output_directory = "./peft_lab_outputs"
training_args = TrainingArguments(
    output_dir=output_directory,
    auto_find_batch_size=True, # find a batch size that fits the size of the data.
    learning_rate= 3e-2,
    num_train_epochs=2,
    report_to="none"
)

trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_sample,
    data_collator=transformers.DataCollatorForLanguageModeling(tokenizer, mlm=False) # this is the dataset ready to be processed in blocks
)
trainer.train()
```

Save and load models

[illegible]

Model inference

```
# Inference
# this function returns the outputs from the model based on the given inputs
def get_outputs(model, inputs, max_new_tokens=100):
    outputs = model.generate(
        input_ids=inputs["input_ids"],
        attention_mask=inputs["attention_mask"],
        max_new_tokens=max_new_tokens,
        repetition_penalty=1.5, #Avoid repetition.
        early_stopping=True, #The model can stop before reach the max_length
        eos_token_id=tokenizer.eos_token_id
    )
    return outputs

input_sentences = tokenizer("I want you to act as a football coach. ", return_tensors="pt")
foundational_outputs_sentence = get_outputs(loader_model, input_sentences.to("cuda"), max_new_tokens=50)

print(tokenizer.batch_decode(foundational_outputs_sentence, skip_special_tokens=True))
```

```
["I want you to act as a football coach. I will provide the information on current events and offer suggestions "
"for actionable tactics that can help improve performance in order of improving team performances. "
"You should also suggest ways how we could increase our players' ability, such as: coaching techniques or other strategies like training"]
```

More insights into LoRA

The role of rank (r) and scaling factor (λ)

r controls the number of trainable parameters.

- A smaller r leads to fewer parameters and less memory consumed.
- A larger r allows to capture more complex task-specific information. Common values are between $r = 4$ and $r = 256$ based on the task complexity and the size of the pre-trained model.

λ controls the magnitude of the weight update.

- It is defined as $\lambda = \frac{\alpha}{r}$, where α is a constant in r and it helps stabilize the training.
- It is usually set to $\lambda = 1$ or $\lambda = 2$ as tuning it is the same as tuning the learning rate of the optimizer.

Where to apply LoRA

LoRA is most effective when applied to the core computational layers of the transformer.

- In the [original paper](#) LoRA is applied only to the query and value projection matrices within the self-attention layer.
- Recent studies demonstrated that applying LoRA to all self-attention matrices and the FFN layers helps.

More recommendations on how to set LoRA's hyperparameters can be found in the [Unsloth's LoRA Hyperparameter Guide](#).

LoRA vs. FullFT

Can LoRA fine-tuning achieve similar performance and sample efficiency as FullFT?

Lora Without Regret (Thinking Machines, 2025)

LoRA can be on par with FullFT but only when a few key factors are chosen correctly:

- 1. Fine-tuning on small-to-medium-sized datasets that do not exceed LoRA's capacity.
- 2. A sufficiently high rank is used to capture the task's complexity. Learning slows down when the adapter runs out of capacity, which is determined by the rank.

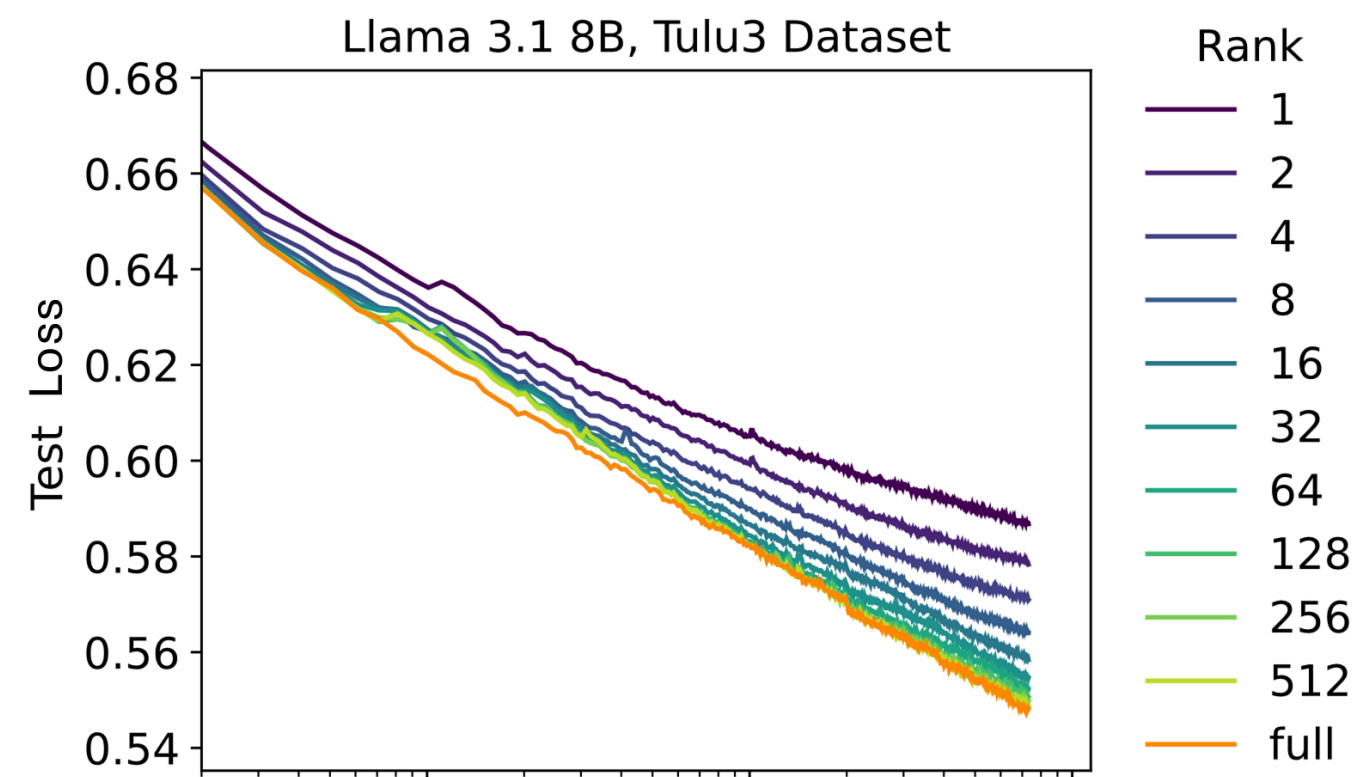


Figure: LoRA training curves for various ranks on Tulu3 and OpenThoughts2 datasets. FullFT and high-rank LoRAs have similar learning curves with loss decreasing linearly with the logarithm of steps. Lower-rank LoRAs fall off the minimum-loss curve when the adapter runs out of capacity.

Source: [Lora Without Regret \(Thinking Machines\)](#)

3. The optimal learning rate for LoRA is higher by a factor of 10 than for FullIFT.

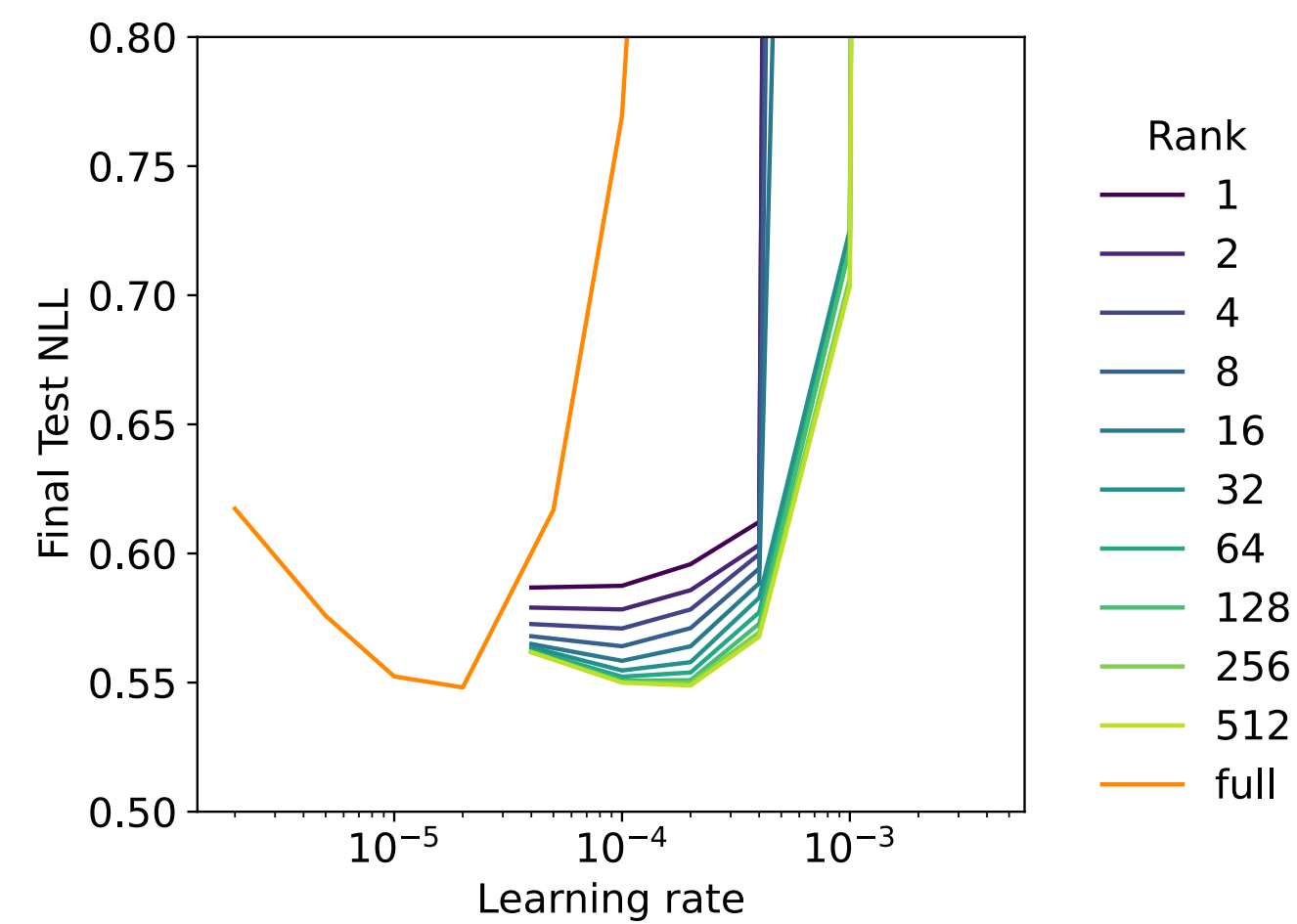


Figure: Learning rate versus final loss for various LoRA ranks on Tulu3. Minimum loss is approximately the same for high rank LoRA and FullIFT. Optimal LR is 10 times higher for LoRA.

Source: [Lora Without Regret \(Thinking Machines\)](#)

4. LoRA is applied broadly, including the attention matrices and the FFN layers.

LoRA vs Full Fine-tuning: An Illusion of Equivalence (Shuttleworth et al., 2024)

The fact that LoRA can match FullFT is only an *illusion* due to insufficient optimization or testing on limited datasets.

- LoRA can match FullFT in simple tasks like sequence classification, instruction tuning, and chat after exact optimization of the hyperparameters.
- LoRA struggles on difficult tasks like code and long-form text generation.
- LoRA and FullFT yield weight matrices with very different structure. FullFT retains most of the pre-training structure while LoRA has a diagonal shift.

LoRA learn less and forget less (Biderman et al., 2024)

FullFT is more accurate and sample-efficient than LoRA, but LoRA forget less of the original knowledge.

- FullFT is more accurate and sample-efficient than LoRA during continuous pre-training, but LoRA matches the FullFT performance with instruction-tuning.
- LoRA forgets less of the original pre-trained knowledge.
- LoRA helps maintaining the diversity of generation.

Conclusion on LoRA vs. FullFT

- It is unclear whether LoRA can match FullFT.
- The recommended solution is to use LoRA in **budget-constrained scenarios!**

Beyond LoRA

QLoRA (Quantized LoRA); DoRA (Weight-Decomposed LoRA); Mixture of LoRA Experts (MoLE); LongLoRA; ConvLoRA; AdaLoRA; QALoRA; ...

- **QLoRA (Quantized LoRA)**: quantizes the pre-trained weights to 4-bit precision while keeping the smaller LoRA matrices in full precision. During backpropagation the original weights are dequantized on-the-fly when needed. This approach achieves a 33% GPU memory saving, at the cost of a 39% increased runtime.
- **DoRA (Weight Decomposed LoRA)**: decouples the pre-trained weights into magnitude and direction. It then applied the LoRA update only to the directional component resulting in a 4% accuracy improvement with minimal added parameters (0.01%).

- **Phi-4-Multimodal**: uses separate dedicated LoRA adapters for different input types e.g., vision and audio. This parameter-efficient strategy allows a single LLM backbone to seamlessly integrate and process complex multimodal inputs without the cost of retraining the entire system for each new modality.

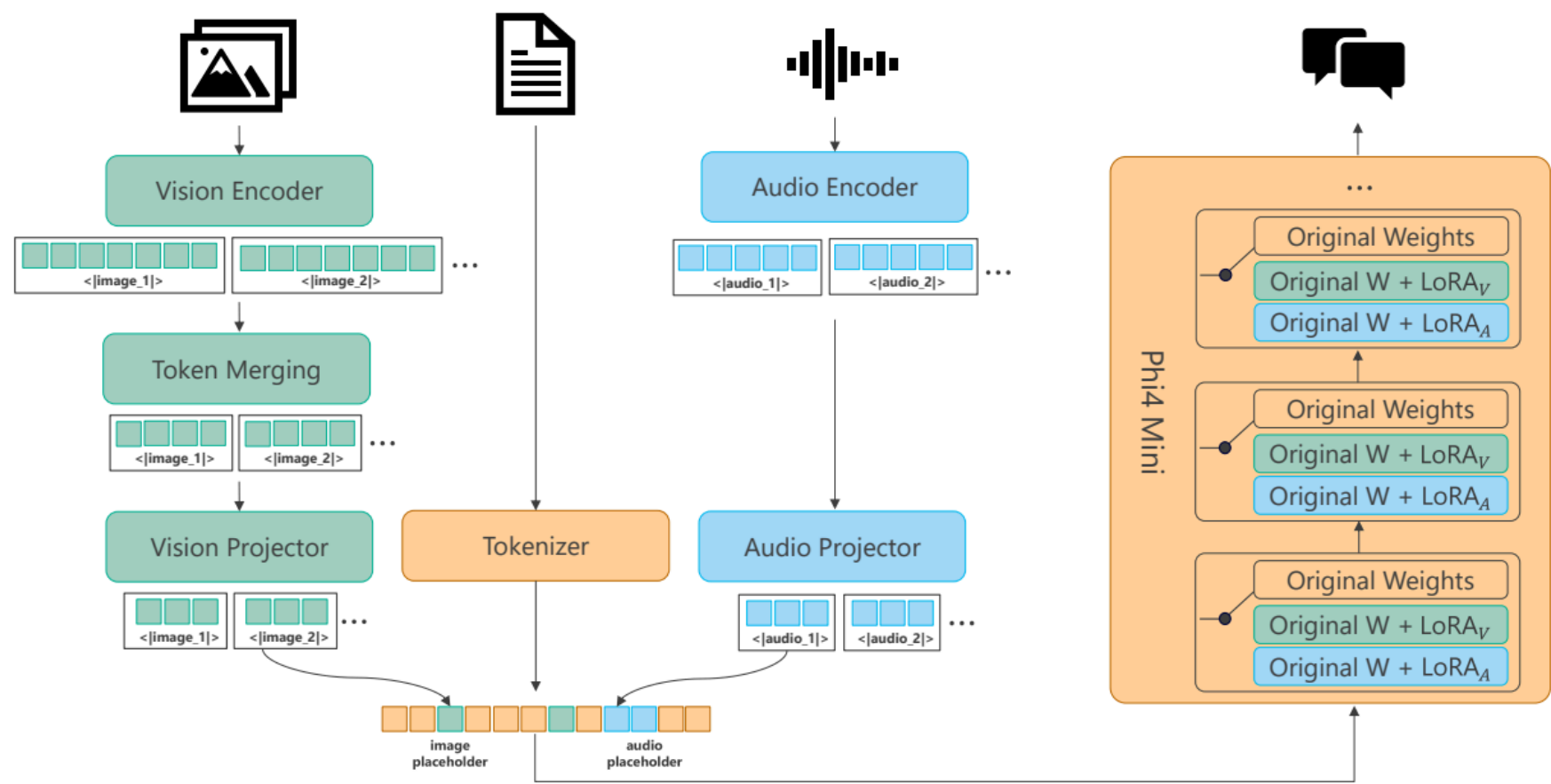


Figure: An overview of the multimodal architecture for Phi-4-Multimodal.

Source: [Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs](#) (Microsoft, 2025)

Useful material:

- Stanford CME295 course. [Transformers & LLMs](#). (Autumn 2025).
- Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- <https://machinelearningmastery.com/encoders-and-decoders-in-transformer-models/>
- Kaplan, Jared, et al. "Scaling laws for neural language models." arXiv preprint arXiv:2001.08361 (2020).
- LoRA and Beyond. Fine-Tuning LLMs for Anyone, Anywhere. <https://amore-labs.github.io/website/notebooks/lora.html> (2025).
- Shuttleworth, Reece, et al. "Lora vs full fine-tuning: An illusion of equivalence." arXiv preprint arXiv:2410.21228 (2024).
- Biderman, Dan, et al. "LoRA Learns Less and Forgets Less." Transactions on Machine Learning Research. (2024)
- Housby, Neil, et al. "Parameter-efficient transfer learning for NLP." ICML (2019).
- Abouelenin, Abdelrahman, et al. "Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-loras." arXiv preprint arXiv:2503.01743 (2025).

- Schulman, John and Thinking Machines Lab, "LoRA Without Regret", Thinking Machines Lab: Connectionism, Sep 2025.
- Practical Tips for Finetuning LLMs Using LoRA (Low-Rank Adaptation). Sebastian Raschka. 2023.
- New LLM Pre-training and Post-training Paradigms. Sebastian Raschka. 2024.
- Unsloth's LoRA Hyperparameter Guide. <https://unsloth.ai/docs/get-started/fine-tuning-llms-guide/lora-hyperparameters-guide>.
- Abouelenin, Abdelrahman, et al. "Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-loras." arXiv preprint arXiv:2503.01743 (2025).
- Dettmers, Tim, et al. "Qlora: Efficient finetuning of quantized llms." NeurIPS (2023): 10088-10115.
- Liu, Shih-Yang, et al. "Dora: Weight-decomposed low-rank adaptation." Forty-first International Conference on Machine Learning. 2024.